/teach me /visualized learning ok continue

Excellent approach. Since you're a beginner and preparing for **Embedded Linux/C/C++ networking interviews**, I'll teach this like a classroom course rather than just giving notes.

We'll use the following learning pattern for every topic:

> **Concept → Visualization → Real-world Example → Data Flow → Linux View → Interview View → Hands-on**

Today we'll start with **Day 2 - Part 1: Understanding the OSI Model**.

Day 2 - OSI Model (Part 1)

Learning Objectives

After this lesson, you should understand:

- Why the OSI model was created
- Why networking needs layers
- Responsibilities of all seven layers
- What happens to data at each layer
- Encapsulation
- Decapsulation
- Protocol Data Units (PDU)

Imagine This Situation

You open Chrome and type:

```
https://www.google.com
```

Then press **Enter**.

Question:

How does your computer know where to send the request?

How does Google know where to send the response?

Why doesn't the request get mixed with millions of other requests?

How does the data travel through Wi-Fi, switches, routers, fiber cables, and finally reach Google's server?

The answer is:

The OSI Model

What is the OSI Model?

OSI stands for

```
Open Systems Interconnection
```

It is a **reference model** created by ISO that explains how two computers communicate over a network.

Think of it as a **step-by-step communication process**.

Instead of one huge networking program doing everything, the work is divided into **seven specialized layers**.

Why Divide into Layers?

Imagine building a car.

Would one engineer do all of these?

- Engine
- Wheels
- Brakes
- Steering
- Paint
- Electronics

No.

Each team specializes.

Networking works the same way.

Each layer has one responsibility.

```
Application Layer
       ↓
Presentation Layer
       ↓
Session Layer
       ↓
Transport Layer
       ↓
Network Layer
       ↓
Data Link Layer
       ↓
Physical Layer
```

Each layer serves the one below it and uses the services of the one below.

Real-Life Example: Sending a Parcel

Suppose you want to send a laptop to your friend.

You don't just throw it onto a truck.

Instead:

```
Laptop
↓

Pack it in a box

↓

Write address

↓

Choose courier

↓

Truck

↓

Road

↓

Friend receives
```

```
↓

Unpack
```

Networking works almost exactly like this.

Your message is packed layer by layer.

OSI Layer Overview

```
+-----------------------------------+
| Layer 7 | Application             |
+-----------------------------------+
| Layer 6 | Presentation            |
+-----------------------------------+
| Layer 5 | Session                 |
+-----------------------------------+
| Layer 4 | Transport               |
+-----------------------------------+
| Layer 3 | Network                 |
+-----------------------------------+
| Layer 2 | Data Link               |
+-----------------------------------+
| Layer 1 | Physical                |
+-----------------------------------+
```

Think of the layers like departments in a company.

Layer	Job
Application	User communication
Presentation	Format data
Session	Manage conversation
Transport	Reliable delivery
Network	Find destination
Data Link	Deliver inside local network
Physical	Send electrical/optical/radio signals

What Happens When You Press Enter?

Let's say you type:

```
GET / HTTP/1.1
Host: google.com
```

Initially, this is **Application Data**.

```
Application Data
```

The Application layer passes it down.

Each lower layer **adds its own information**.

This process is called:

Encapsulation

Think of putting a letter into multiple envelopes.

```
Letter

↓

Envelope

↓

Courier Label

↓

Truck Container

↓

Truck
```

Networking is the same.

Visualization of Encapsulation

```
Application Layer

Creates:

---------------------------------
GET / HTTP/1.1
```

```
---------------------------------

↓

Presentation Layer

May compress or encrypt

---------------------------------
Encrypted HTTP Request
---------------------------------

↓

Session Layer

Adds session information

---------------------------------
Session + Encrypted Data
---------------------------------

↓

Transport Layer

Adds TCP Header

-----------------------------------------
TCP Header | Session Data
-----------------------------------------

↓

Network Layer

Adds IP Header

-------------------------------------------------
IP Header | TCP Header | Session Data
-------------------------------------------------

↓

Data Link Layer

Adds Ethernet Header

Adds Ethernet Trailer (CRC)

-----------------------------------------------------------------------
Ethernet Header | IP Header | TCP Header | Session Data | Ethernet CRC
```

```
------------------------------------------------------------------------

↓

Physical Layer

Converts into bits
```

Notice something important.

The original HTTP request is still inside.

Every layer wraps the previous layer.

Like nesting boxes.

Encapsulation Analogy

Imagine a birthday gift.

```
Gift

↓

Gift Box

↓

Shipping Box

↓

Courier Label

↓

Truck
```

```
Truck
└── Shipping Box
      └── Gift Box
            └── Gift
```

Networking works exactly like this.

Each layer wraps the previous layer.

Data Units (PDU)

As data moves down the stack, its name changes.

```
Application

↓

Data

↓

Transport

↓

Segment (TCP)

or

Datagram (UDP)

↓

Network

↓

Packet

↓

Data Link

↓

Frame

↓

Physical

↓

Bits
```

Interviewers frequently ask:

> **Why does the name change?**

Because each layer adds different control information, making the unit suitable for its own function.

What Does Each Layer Add?

```
Application

"My message"

↓

Transport

"I'll deliver it reliably."

↓

Network

"I know where it should go."

↓

Data Link

"I know the next device on the local network."

↓

Physical

"I'll convert it into electrical/light/radio signals."
```

Each layer has a different responsibility.

The Receiver Does the Opposite

When Google's server receives the data:

```
Bits

↓

Frame
```

```
↓

Packet

↓

TCP Segment

↓

Application Data
```

This is called

Decapsulation

Visualization:

```
Received

Ethernet Frame

↓

Remove Ethernet Header

↓

IP Packet

↓

Remove IP Header

↓

TCP Segment

↓

Remove TCP Header

↓

Original HTTP Request

GET / HTTP/1.1
```

Every layer removes only **its own header** and passes the rest upward.

Why Don't Layers Remove Everything?

Each layer only understands its own protocol.

For example:

- Layer 2 understands Ethernet.
- Layer 3 understands IP.
- Layer 4 understands TCP or UDP.

This modular design allows protocols to evolve independently.

Data Journey Summary

```
User Types

↓

HTTP Request

↓

Application Layer

↓

Presentation Layer

↓

Session Layer

↓

Transport Layer

↓

Network Layer

↓

Data Link Layer

↓

Physical Layer

↓↓↓↓↓↓↓↓↓↓↓↓↓↓↓↓↓

Network
```

```
↓↓↓↓↓↓↓↓↓↓↓↓↓↓↓↓↓

Physical

↓

Data Link

↓

Network

↓

Transport

↓

Session

↓

Presentation

↓

Application

↓

Google Receives HTTP Request
```

Why Is This Useful?

Suppose a webpage doesn't load.

Which layer is failing?

Examples:

- Wi-Fi cable unplugged → Physical layer
- Wrong MAC communication on local network → Data Link layer
- Incorrect IP or routing → Network layer
- TCP connection not established → Transport layer
- HTTPS handshake issue → Presentation layer

- Browser cannot create the request → Application layer

The OSI model helps isolate and troubleshoot problems systematically.

Interview Questions

1. What is the OSI model?

A seven-layer reference model describing how data moves between networked systems.

---

2. Why was the OSI model created?

To standardize communication, simplify design, improve interoperability, and make troubleshooting easier by separating networking into layers.

---

3. What is encapsulation?

The process of adding layer-specific headers (and, at Layer 2, a trailer) as data moves from the Application layer to the Physical layer.

---

4. What is decapsulation?

The reverse process where each receiving layer removes its own header (or trailer) before passing the remaining data to the next higher layer.

---

5. Why do we need layers?

Because each layer focuses on a specific responsibility, making networks easier to develop, maintain, troubleshoot, and upgrade.

Visual Memory Trick

```
A
P
S
T
N
D
P
```

Meaning:

```
Application

Presentation

Session

Transport

Network

Data Link

Physical
```

Or remember the sentence:

> **"All People Seem To Need Data Processing."**

What You'll Learn Next (Day 2 - Part 2)

We'll take a deep dive into the **top three layers**:

- **Application Layer** — How browsers, email clients, SSH, DNS, and other applications generate network data.
- **Presentation Layer** — Data formats, character encoding (ASCII/UTF-8), compression, encryption (TLS/SSL), and why HTTPS is secure.
- **Session Layer** — Session establishment, maintenance, synchronization, and termination, with real examples such as logging into websites and remote SSH sessions.

We'll also trace **exactly what happens inside your browser** from the moment you press **Enter** until the request reaches the Transport layer, with detailed diagrams and Linux-relevant interview explanations.